

Ejemplo 1: Usando una Pila para Deshacer Acciones (Ctrl + Z)

En muchas aplicaciones de edición de texto o gráficos, se utiliza una pila para manejar la funcionalidad de deshacer. Cada vez que haces una acción (como escribir o dibujar), se guarda en una pila. Cuando presionas Ctrl + Z, se "deshace" la acción más reciente que se encuentra en la parte superior de la pila.

```
import java.util.Stack;

public class EditorTexto {
    public static void main(String[] args) {
        Stack<String> acciones = new Stack<>();

        // Simular acciones de un editor de texto
        acciones.push("Escribir 'Hola'");
        acciones.push("Escribir 'Mundo'");
        acciones.push("Borrar 'Mundo'");

        // Deshacer última acción
        System.out.println("Acción a deshacer: " + acciones.pop()); // Borrar 'Mundo'
        System.out.println("Acción a deshacer: " + acciones.pop()); // Escribir 'Mundo'
    }
}
```

push() agrega una acción a la pila.

pop() remueve la última acción y la devuelve, simulando la operación de deshacer.

Ejemplo 2: Usando una Cola para Simular una Fila en un Banco

En una cola de banco o en un supermercado, los clientes son atendidos en el orden en que llegan (primero en entrar, primero en salir). Esto se puede implementar con una cola (FIFO: First In, First Out).

```
import java.util.LinkedList;
import java.util.Queue;

public class FilaBanco {
    public static void main(String[] args) {
        Queue<String> filaClientes = new LinkedList<>();

        // Clientes que llegan a la fila
        filaClientes.add("Cliente 1");
        filaClientes.add("Cliente 2");
        filaClientes.add("Cliente 3");

        // Atender a los clientes en el orden que llegaron
        while (!filaClientes.isEmpty()) {
            System.out.println("Atendiendo a: " + filaClientes.poll());
        }
    }
}
```

Add() añade un cliente a la cola.

poll() remueve al cliente que llegó primero para atenderlo.

Diferencias entre Pila y Cola:

- Pila (Stack): Sigue el principio LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir. Es útil para manejar tareas donde el orden inverso es importante, como deshacer acciones o retroceder en la navegación web.
- Cola (Queue): Sigue el principio FIFO (First In, First Out), donde el primer elemento en entrar es el primero en salir. Es ideal para situaciones en las que el orden de llegada es importante, como en colas de atención o procesamiento de tareas.

CONSTRUCTORES

Paso 1: Entender la clase y sus atributos (Definir los atributos que quieres inicializar.)

Antes de crear un constructor, necesitamos saber qué atributos (variables) tiene la clase. Estos son los valores que el constructor inicializará cuando se cree un objeto de la clase.

Ejemplo:

Vamos a suponer que estamos diseñando una clase llamada Coche con los siguientes atributos:

*marca (tipo String)

*modelo (tipo String)

*año (tipo int)

Código:

```
class Coche {
    String marca;
    String modelo;
    int año;
}
```

Paso 2: Definir el constructor (Crear el constructor con el mismo nombre que la clase, sin tipo de retorno.)

Un constructor debe tener el mismo nombre que la clase y no puede tener un tipo de retorno.

Su función es inicializar los atributos cuando se crea un objeto de la clase.

2.1 Crear un constructor sin parámetros:

Podemos crear un constructor vacío (sin parámetros) que asigna valores predeterminados a los atributos.

```
class Coche {
    String marca;
    String modelo;
    int año;

    // Constructor sin parámetros
    public Coche() {
        // Asignar valores predeterminados
        this.marca = "Desconocida";
        this.modelo = "Desconocido";
        this.año = 0;
    }
}
```

Constructor Coche(String marca, String modelo, int año): Este constructor recibe tres parámetros, que son asignados a los atributos de la clase mediante this.marca, this.modelo, y this.año.

2.2 Crear un constructor con parámetros:(Definir parámetros en el constructor si deseas pasar valores para inicializar los atributos)

Para permitir que el constructor inicialice los atributos con valores específicos al crear un objeto, necesitamos añadir parámetros al constructor. (Usar this para referirte a los atributos del objeto actual y diferenciarlos de los parámetros.)

```
class Coche {
    String marca;
    String modelo;
    int año;
```

```

// Constructor con parámetros
public Coche(String marca, String modelo, int año) {
    // Usamos 'this' para referirnos a los atributos de la clase
    this.marca = marca;
    this.modelo = modelo;
    this.año = año;
}
}

```

Paso 3: Usar el constructor en el método main

Para usar el constructor, necesitamos crear una instancia de la clase Coche en el método main. Esto se hace usando la palabra clave new, que invoca al constructor y crea un nuevo objeto.

```

public class Main {
    public static void main(String[] args) {
        // Crear un objeto de la clase Coche usando el constructor con parámetros
        Coche miCoche = new Coche("Toyota", "Corolla", 2020);

        // Acceder a los atributos del objeto
        System.out.println("Marca: " + miCoche.marca);
        System.out.println("Modelo: " + miCoche.modelo);
        System.out.println("Año: " + miCoche.año);
    }
}

```

Método main: Aquí se crea una instancia de Coche llamada miCoche, y el constructor asigna los valores "Toyota", "Corolla" y 2020 a los atributos de la instancia.
(Invocar el constructor usando new al crear una instancia de la clase.)

Paso 4: Entender la palabra clave this

La palabra clave this se utiliza dentro de un constructor (o método) para hacer referencia al objeto actual.

Esto es necesario cuando los nombres de los parámetros tienen el mismo nombre que los atributos de la clase.

```

// Clase que representa un nodo de la lista enlazada
class Nodo {
    int dato;
    Nodo siguiente;

    public Nodo(int dato) {
        this.dato = dato;
        this.siguiente = null; // El siguiente nodo es nulo inicialmente
    }
}

// Clase que representa la lista enlazada
class ListaEnlazada {
    Nodo cabeza; // Puntero (referencia) al primer nodo de la lista

    public ListaEnlazada() {
        this.cabeza = null; // Al principio, la lista está vacía
    }

    // Método para añadir un nodo al final de la lista
    public void agregar(int dato) {
        Nodo nuevoNodo = new Nodo(dato);

        if (cabeza == null) {
            cabeza = nuevoNodo; // Si la lista está vacía, el nuevo nodo será la cabeza
        } else {
            Nodo temp = cabeza;
            while (temp.siguiente != null) {
                temp = temp.siguiente; // Recorre la lista hasta el final
            }
            temp.siguiente = nuevoNodo; // Añade el nuevo nodo al final
        }
    }

    // Método para imprimir los elementos de la lista
    public void imprimirLista() {
        Nodo temp = cabeza;
        while (temp != null) {
            System.out.print(temp.dato + " ");
            temp = temp.siguiente; // Avanza al siguiente nodo
        }
    }
}

// Clase principal para probar la lista enlazada
public class Main {
    public static void main(String[] args) {
        ListaEnlazada lista = new ListaEnlazada();

        // Añadir elementos a la lista
        lista.agregar(10);
        lista.agregar(20);
        lista.agregar(30);
        lista.agregar(40);
        lista.agregar(50);

        // Imprimir la lista
        lista.imprimirLista(); // Salida: 10 20 30
    }
}

```